

CS 415 Operating Systems Spring 2026

Project #3 Report Collection

Author:

Lucas Lee Bixby

UO ID: lbixby

952001359

Report

Introduction:

In this project, I was tasked with creating a three-part implementation to simulate an amusement park as an exploration of using threads. Part 1 contained a single-thread solution for both car and passenger threads, such that the implementation could be tested without the use and complexity of using a mutex, and just relying on signal passing for state tracking. Part 2 then expanded upon Part 1, implementing a multithreading implementation with command-line input for simulation parameters. This allowed for multiple car and passenger threads to run concurrently throughout the system, all while logging thread states every second the simulation ran. In Part 3, an additional layer of tracking was added on top of the code base from Part 2 to provide insights into the stats of each element in the park - displaying once every five seconds, and once at the end when the park closes.

This project was vital in understanding the life cycle of threads, along with how to handle the critical section problem and race conditions using semaphores, mutexes, and signal handling.

Background:

Consistent project architecture played a vital role in maintaining organization and efficiency as I scaled my project to add more functionality. I managed this task by creating a helper file named `helpers.c`, which contained all my helper functions, and the header file `park.h`, which linked each function to the main part of each project (`park.c`). On top of this, I also created a GitHub repository to manage the version control and commit history of my project, for organization, and practice with professional industry tools. These combined methods made scaling my code efficient and clean. When I needed to start the next part of my project after completing a section, I would copy over the previous code into a new directory and edit the former code base to match the specifications for each part of the assignment. Additionally, to add another layer of organizational methods, after working on the project for a while, I would write myself development notes as a comment in the header of the code I was working on, to preserve my train of thought for when I returned. Additionally, I included thorough commenting throughout my code to make it easier to identify the functionality and purpose of my code when it came to debugging, scaling, implementing, and navigating my codebase.

Implementation:

As for the general structure and approach that I used to solve the tasks for parts 1-3. Each worker thread (passenger, car, and monitor) had a function that defined their behavior; these functions would continuously execute while the park was open.

*For **passenger threads**, their behavior followed:*

Explore park → Get ride ticket → Enter ride queue → Board car → Unboard car

This allowed for the passenger threads to have a pattern and tasks to follow while interacting within the simulation. Even though there was a guide of objectives for each thread, the outputs of their statuses were not uniform due to several different variabilities, such as random intervals of time (1-10), exploring the park, waiting in ticket/ ride queues, and ticket processing time. These random and specified processing times allowed for the simulation to run uniquely and realistically every time, along with staggering the order of execution for each passenger thread.

For **car threads**, their specified behavior followed:

Car load → Car run → Car unload

This made it easy to allow the car threads to wait for certain conditions defined by its state before executing (such as load, unload, and car_ready). Additionally, my implementation only allowed for one car to load at a time, so the only time passengers would be waiting in the ride queue was if there were no cars at the loading doc, the current car was full, or the cars were unloading.

Finally, for the **Monitor thread** behavior (for part 3), followed:

Sleep for 5 seconds → Check if park is still open → Output system log → Repeat

Creating an individual thread for the monitor logging system ended up serving as a great advantage by allowing it to work in the background of the simulation. Keeping track of the park status and executing its logging function calls on its own time, while blocking the printing mutex to take priority over the designated output. Rather than trying to place system logs within the code of other functions, it constantly checks the time from which the park has opened, relying heavily on external variables.

Additionally, one interesting and clever choice that I made in my implementation of the project was within the data collection for the system results. I ended up creating and allocating physical queues for the line and ticket queues to track the status of the passengers waiting in line. Along with this choice came creating thread-safe enqueue and dequeue helper functions. I particularly thought that this was an interesting approach because of the applicability of this common problem in real-world software.

Below is my implementation of the **enqueue** and **dequeue** helper functions:

```

96
97     void enqueue(pthread_mutex_t queue_type, int *queue, int size, int id)
98     // helper function for thread safe queueing
99     {
100         pthread_mutex_lock(&queue_type);
101         queue[size] = id;
102         pthread_mutex_unlock(&queue_type);
103     }
104
105     int dequeue(pthread_mutex_t queue_type, int *queue, int size)
106     // helper function for thread safe dequeuing
107     {
108         pthread_mutex_lock(&queue_type);
109         int removed = queue[0];
110         for (int i = 0; i < size - 1; i++) {
111             queue[i] = queue[i + 1];
112         }
113         queue[size - 1] = -1;
114         pthread_mutex_unlock(&queue_type);
115         return removed;
116     }
117

```

Performance Results and Discussion

The performance of my project is up to par in terms of features, memory allocation, and output. I developed my assignment for Project_3 through an SSH connection to the University of Oregon's ix-dev system, and there were no noticeable performance issues when running tests or the final program. Although I didn't have the time to optimize my code to peak its performance, my provided architecture matches the assignment details and specifications. As for memory allocation and clean up, I ran my program with Valgrind to check for memory leaks; in all cases for parts 1-3, they pass (valgrind logs can be found inside the output directories within each part 1-3 directory of my submission).

Conclusion

Overall, this project was really interesting in how we utilized condition signals, mutex/semaphores to manage the flow of threads within our simulation, and work around the critical section problem for race conditions. I definitely learned a lot of useful skills and techniques that gave me a deeper understanding of how to properly utilize threading within a program. I think that if I were to come back and revisit this project in the future. I would experiment with batching cars to load and run at the same time to optimize the total throughput of passengers.